# Lagrange Interpolation

## 1. 문제 정의

- Interpolation (보간)

주어진 데이터 점들을 정확히 지나는 함수를 만들어

그 사이의 값을 추정하는 방법

=> 서로 다른 n+1개의 점

$(x_0, f(x_0)), (x_1, f(x_1)) \cdots, (x_n, f(x_n))$이 있을때

이 점들을 "모두" 지나는 차수 ≤n인 다항식

$P_n$ 찾는 것이 목표

※ 이런 다항식은 존재하고 유일하다

## 2. 공식

아이디어 : 각 점에서만 1, 나머지 노드에서는 0되는 다항식

$$L_{n,k}(x) = \frac{(x-x_0)\cdots(x-x_{k-1})(x-x_{k+1})\cdots(x-x_n)}{(x_k-x_0)\cdots(x_k-x_{k-1})(x_k-x_{k+1})\cdots(x_k-x_n)}$$

이 다항식은 $L_{n,k}(x_i)=0 \ (i \neq k)$, $L_{n,k}(x_k)=1$

만족 => 즉, k번째 data만 선택하는 게 가능

따라서 전체 보간 식

$$P(x) = \sum_{k=0}^{n} f(x_k) L_{n,k}(x)$$

## 3. linear 라그랑주

점 2개 $(x_0, y_0)$, $(x_1, y_1)$라면

$$P(x) = \frac{x-x_1}{x_0-x_1} f(x_0) + \frac{x-x_0}{x_1-x_0} f(x_1)$$

점이 3개 => 2차다항식

4개 => 3차 〃

$x_0, x_1, x_2$

=> $P(x) = f(x_0)L_0(x) + f(x_1)L_1(x) + f(x_2)L_2(x)$

## 4. Pros / Cons

Pros — 직관적

Cons — 노드 추가 = 식 다시... 재사용 힘듦

=> divided difference로 연결

# Divied Difference

## 1. 문제 정의

라그랑주 보간의 계산 비효율성 극복

=> 재귀적 구성, 갱신 쉽게 Newton form으로 표현

## 2. 분할차분 기본 정의

0차 분할차분 = 해당 점 함숫값

$$f[x_i] = f(x_i)$$

(최저차수)

1차 분할차분 = 두 점 사이 기울기

$$f[x_i, x_{i+1}] = \frac{f(x_{i+1}) - f(x_i)}{x_{i+1} - x_i}$$

고차 분할차분 (k차)

= 바로 아래 단계 (k-1차) 분할차분 2개 차이를 양 끝 노드의

차이로 나눈 값

$$f[x_i, \cdots, x_{i+k}] = \frac{f[x_{i+1}, \cdots, x_{i+k}] - f[x_i, \cdots, x_{i+k-1}]}{x_{i+k} - x_i}$$

## 3. 뉴턴 form 보간 다항식

$$P_n(x) = a_0 + a_1(x-x_0) + a_2(x-x_0)(x-x_1) \cdots + a_n(x-x_0)\cdots(x-x_{n-1})$$

이때 계수 $(a_0 \cdots a_n)$이 앞에서 구한 분할 차분 값

$a_0 = f[x_0]$, $a_1 = f[x_0, x_1]$ ... $a_n = f[x_0, x_1, \cdots, x_n]$

이 식이 분할차분의 목적, 추가노드가 있으면 $a_{n+1}$ ... 처럼

한 항만 붙이면 됨!

Min

# Neville's

## 1. 개념

보간다항식을 전개해 식 찾기 X

→ 낮은 차수 근사값들 조합 => 더 정확한 근사값 찾기

(특정 점 $x$에서의 근사값 안정적으로 계산)

## 2. 아이디어

작은 차수의 보간 합쳐서 큰 차수 보간 재귀적 생성

## 3. 표기

인덱스 복잡(헷갈...) 방지 위해 $Q_{ij}(x)$ 사용

$i$ (행) : 사용된 데이터 노드 마지막 인덱스

$j$ (열) : 다항식 차수

초기값 = $Q_{i,0}(x) = f(x_i)$ = 함숫값

## 4. 재귀식

$$Q_{ij}(x) = \frac{(x - x_{i-j})Q_{i,j-1}(x) - (x - x_i)Q_{i-1,j-1}(x)}{x_i - x_{i-j}}$$

현재 값 $Q_{i,j}$를 구하기 위해 왼쪽값 $(Q_{i,j-1})$, 대각왼쪽 위 값 $(Q_{i-1,j-1})$ 가져와서 더 높은 차수 보간값 만듦

## 5. 장점

1. 계산 효율 : 특정 점에서의 값 하나만 필요할때 유리

라그랑주처럼 모든 식 X, 뉴턴처럼 모든 계수 X

# 보간/외삽, 오차

## 1. interpolation vs extrapolation

- inter-~ (보간) : 주어진 노드들 사이의 값 추정
- Ex ~ (외삽) : 주어진 노드 범위 밖의 값을 추정

보간이 상대적으로 안정적

## 2. 오차공식

라그랑주 보간 $P_n(x)$의 오차

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!}\prod_{i=0}^{n}(x - x_i)$$

=> 오차식 해석

1. $f^{(n+1)}(\xi)$ : 함수의 굴곡 = 고차 미분값

굴곡이 클수록 오차↑

2. $\prod_{i=0}^{n}(x - x_i)$ : 평가점 $x$와 노드들과의 거리

작으면 오차↓, 크면 ↑

=> 노드 사이 "내부" 점에서는 오차↓,

노드 범위 밖은 오차↑

따라서 외삽은 범위 밖이므로 오차가 크다.

## 3. 보간에서 봐야할 것

(1) 평가점이 노드들 사이 인지

보간이면 ok, 외삽이면 불리함

(2) 평가점과 가까운 노드 사용

=> 오차↓

(3) 차수의 함정

차수 높을수록 정확 X

ex) $f(x) = \frac{1}{x}$ 을 $x_0 = 1$ 근처에서 전개, $x = 3$

근사하면 $1, -1, 3, -5, \cdots, -21$ 처럼 더

나빠질 수도 ...

Min